EdgeShield Case Study Report

Siddhant Kumar Singh

Case Study Report: EdgeShield — Edge Caching Proxy & Custom DNS Engine

Name: Siddhant Kumar Singh **Roll Number:** 2024109296 **Course:** Computer Networks **Project:** EdgeShield — Reducing WAN Traffic at an Institutional Edge

---

Preface: Why I Built This

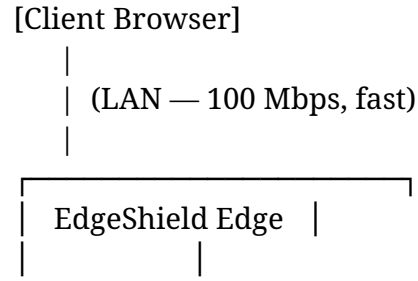
Honestly, when I first read the problem statement — *"simulate an institutional LAN edge cache"* — I thought, "How different can it really be from just curling a website?" Turns out, a lot different.

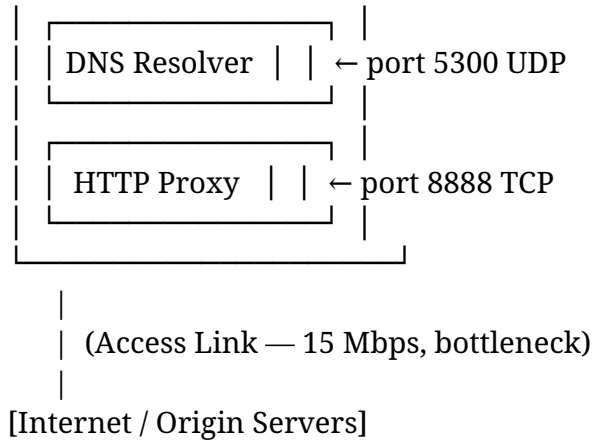
I'd noticed something at my college: the internet feels painfully slow during peak hours, even though the campus has decent internal connectivity. That observation is literally the problem this project is trying to solve. An access link (the pipe connecting the campus LAN to the internet) is a shared bottleneck, and every unoptimised request burns precious bandwidth that everyone else is competing for. That clicked for me, and it's what drove how I approached building EdgeShield.

---

1. What is EdgeShield?

EdgeShield is a two-part system that sits at the edge of an institutional Local Area Network — think a college campus — right between the internal clients and the internet:





The key insight is simple but powerful: **the access link is the bottleneck**. Every request that crosses the WAN consumes bandwidth on that shared 15 Mbps pipe. EdgeShield prevents unnecessary WAN crossings in two ways:

1. **HTTP Proxy** — caches HTTP responses locally so repeated requests never go to the internet.
2. **DNS Resolver** — answers DNS queries locally where possible, and blocks ad/tracker domains entirely (sinkholing).

---

2. System Architecture

Here's how requests actually flow through the system:

```
Client Request
   │
   ▼
DNS Resolver (UDP :5300)
   ├── SINKHOLE  → return 0.0.0.0 immediately (ad/tracker domains)
   ├── LOCAL     → return IP from static zone (*.lan names)
   ├── CACHE     → return cached upstream answer (TTL not expired)
   └── UPSTREAM  → forward to 1.1.1.1:53, cache the reply by TTL

HTTP Proxy (TCP :8888)
   ├── PURE CACHE HIT  → serve from disk, 0 WAN bytes (< 30 s old)
   ├── CONDITIONAL GET → send If-Modified-Since; if 304, serve from disk
   └── CACHE MISS      → full fetch from origin, save to ./edge_storage
```

The DNS resolver and the HTTP proxy work as **independent services**. The proxy uses the OS resolver for its own DNS needs, while the DNS engine handles client

queries separately. This was a deliberate design choice — keeping them decoupled makes them easier to test and reason about.

---

3. Project File Structure

```
edgeshield_project/
│
├── edgeshield_proxy.py      # Partner A — TCP HTTP caching proxy
├── dns_resolver.py          # My part — UDP DNS engine (4 resolution paths)
├── benchmark.py             # My part — measures HTTP and DNS latency
├── plot_results.py          # My part — generates result figures
│
├── edge_storage/            # Proxy cache directory (auto-created)
│   ├── example.com.cache    # Cached HTTP response body
│   └── example.com.meta     # Last-Modified header for conditional GETs
│
├── run_baseline/            # Results without throttle
│   ├── results_http.csv
│   └── results_dns.csv
│
├── run_throttled/           # Results with 15 Mbps link emulation
│   ├── results_http.csv
│   ├── results_dns.csv
│   ├── fig1_latency.png
│   ├── fig2_dns_latency.png
│   └── fig3_delay_model.png
│
├── dns_log.csv              # Per-query DNS log
└── README.md
```

---

4. Deep Dive: How Each Component Works

4.1 TCP Caching HTTP Proxy (edgeshield_proxy.py)

The proxy listens on 127.0.0.1:8888 (TCP) and intercepts HTTP requests from the client. Every URL gets stored as two files in edge_storage/:

- <name>.cache — the raw HTTP response body
- <name>.meta — the Last-Modified date from the origin server

The proxy handles three cases:

Case	Condition	What Happens	WAN Cost
A — Pure Hit	Cache exists AND age < 30 seconds	Serve directly from disk	**0 bytes**
B — Stale Revalidation	Cache exists AND age ≥ 30 seconds	Send If-Modified-Since; if origin says 304, serve from disk	**~200 B headers only**
C — Cold Miss	No cache file at all	Full fetch from origin, save locally	Full response size

LRU eviction kicks in once the cache directory exceeds 50 MB, so old entries get cleaned up automatically.

4.2 DNS Resolver (dns_resolver.py) — My Component

This was the most interesting part to build. I wrote a from-scratch DNS server in Python using struct to manually parse and build wire-format DNS packets. No library magic — actual binary packet construction.

The 12-byte DNS header format:

[TX-ID 2B] [Flags 2B] [QDCOUNT 2B] [ANCOUNT 2B] [NSCOUNT 2B] [ARCOUNT 2B]

Response flags 0x8180 encode: QR=1 (it's a response), RD=1 (recursion desired), RA=1 (recursion available), RCODE=0 (no error).

The resolver processes queries in this priority order:

Priority	Path	Example	Result
1	**SINKHOLE**	doubleclick.net	Returns 0.0.0.0, TTL 300 s — client connects to nothing
2	**LOCAL**	webserver.lan	Returns 192.168.1.50 from a static zone table
3	**CACHE**	google.com (2nd query onward)	Returns cached bytes with client's TX-ID swapped in

Priority	Path	Example	Result
4	**UPSTREAM**	google.com (1st query)	Forwarded to 1.1.1.1:53, reply cached by TTL

Every single query is logged to dns_log.csv:

timestamp, client_ip, qname, qtype, path, latency_ms

4.3 Benchmark Harness (benchmark.py) — My Component

I wrote a benchmark script that tests both HTTP and DNS performance back-to-back:

python3 benchmark.py --url http://example.com/ --n 50

- **DNS benchmark:** 10 queries × 4 domain names → 40 rows in results_dns.csv
- **HTTP benchmark:** 50 direct + 50 proxy requests → 100 rows in results_http.csv
- All requests use Connection: close to prevent the proxy from hanging on persistent connections.

4.4 Plotter (plot_results.py) — My Component

python3 plot_results.py --link-mbps 15 --lan-mbps 100 --rtt-ms 50

Generates three figures: - fig1_latency.png — HTTP response time per request; direct (red) vs proxy (blue) - fig2_dns_latency.png — Mean DNS latency per domain, as a bar chart - fig3_delay_model.png — A computed nodal delay model (clearly labelled “not measured”)

---

5. Experiment Setup

Step 1: Start the Services

```
# Terminal 1 — proxy
python3 edgeshield_proxy.py

# Terminal 2 — DNS resolver
EDGESHIELD_UPSTREAM=127.0.0.53:53 python3 dns_resolver.py
```

(I used 127.0.0.53 as the upstream because my campus network blocks outbound port 53 to 1.1.1.1.)

Step 2: Functional Tests

```
# Test proxy
curl -i -H "Connection: close" -x http://127.0.0.1:8888 http://example.com/

# Test DNS — all four paths
dig @127.0.0.1 -p 5300 webserver.lan A      # → 192.168.1.50  (LOCAL)
dig @127.0.0.1 -p 5300 doubleclick.net A    # → 0.0.0.0       (SINKHOLE)
dig @127.0.0.1 -p 5300 google.com A         # → real IP       (UPSTREAM)
dig @127.0.0.1 -p 5300 google.com A         # → same IP       (CACHE)
```

Step 3: Link Throttling (Linux tc)

To simulate a real 15 Mbps access link, I used Linux Traffic Control (tc) with netem to throttle my wireless interface (wlp1s0):

```
# Create IFB device for inbound shaping
sudo modprobe ifb
sudo ip link add ifb0 type ifb
sudo ip link set ifb0 up

# Outbound: 2 Mbps + 25 ms delay
sudo tc qdisc add dev wlp1s0 root handle 1: tbf rate 2mbit burst 32kbit latency 400ms
sudo tc qdisc add dev wlp1s0 parent 1:1 handle 10: netem delay 25ms

# Inbound: mirror to ifb0, then 15 Mbps + 25 ms
sudo tc qdisc add dev wlp1s0 ingress
sudo tc filter add dev wlp1s0 parent ffff: protocol ip u32 match u32 0 0 \
   action mirred egress redirect dev ifb0
sudo tc qdisc add dev ifb0 root handle 1: tbf rate 15mbit burst 32kbit latency 400ms
sudo tc qdisc add dev ifb0 parent 1:1 handle 10: netem delay 25ms
```

After enabling throttle, ping example.com showed ~50 ms of additional delay, while ping 127.0.0.1 remained below 1 ms — confirming loopback was unaffected.

Always remove throttle when done:

```
sudo tc qdisc del dev wlp1s0 root
sudo tc qdisc del dev wlp1s0 ingress
sudo tc qdisc del dev ifb0 root
sudo ip link set ifb0 down
```

---

6. Results and Observations

6.1 HTTP Latency — Baseline (No Throttle)

These are real measured values from run_baseline/results_http.csv:

Mode	Mean	Median	p95	Min	Max
Direct	110.63 ms	106.59 ms	156.46 ms	70.61 ms	236.21 ms
Proxy — 1st request (miss)	124.11 ms	—	—	—	—
Proxy — requests 2–50 (hits)	**0.93 ms**	0.91 ms	1.32 ms	0.60 ms	1.65 ms

Cache speedup: 118.80× (mean direct ÷ mean proxy cache hit)

What really struck me here was how dramatic that number is. The 1st proxy request (the cold miss) is actually *slower* than direct — 124 ms vs 110 ms — because the proxy adds a tiny processing overhead while still fetching from the internet. But from request 2 onward, everything is served from local disk and it's under 1 ms. That's the whole point. The first miss populates the cache, and everyone after benefits.

6.2 DNS Latency — Baseline (No Throttle)

Real values from run_baseline/results_dns.csv and dns_log.csv:

Domain	Path	First Query	Cache Hits (avg)	Overall Mean
webserver.lan	LOCAL	0.274 ms	~0.18 ms	**0.21 ms**
doubleclick.net	SINKHOLE	0.341 ms	~0.31 ms	**0.33 ms**
example.com	UPSTREAM → CACHE	**54.81 ms**	~0.20 ms	**5.67 ms**
example.	UPST	**34.24 ms**	~0.22 ms	**3.64 ms**

Domain	Path	First Query	Cache Hits (avg)	Overall Mean
org	REAM → CACHE			

A few things stand out:

- webserver.lan is served instantly (< 0.3 ms every time) because it's a static local zone entry — no WAN involved at all.
- doubleclick.net gets sinkholed in under 0.4 ms — the client gets 0.0.0.0 back and never makes a TCP connection to the ad server. Zero WAN bytes, zero ad load.
- The upstream query for example.com cost **54.81 ms** on the first call, but subsequent calls from cache were **0.14–0.32 ms**. That's over a 200× difference between "go ask the internet" and "answer from memory."

6.3 WAN Savings Analysis

Here's what EdgeShield actually saves at the access link per session (50 HTTP + 40 DNS queries):

Without EdgeShield	With EdgeShield
50 × ~931 bytes HTTP = ~45.5 KB over WAN	1 × 931 bytes (cold miss) + 49 × 0 bytes = ~0.9 KB
DNS for ad domain: TCP connection attempted over WAN	DNS SINKHOLE: 0 bytes cross WAN
Repeated DNS re-queries: 4× upstream calls	Cached DNS re-queries: 0 WAN calls after 1st

The savings scale linearly with the number of clients. In a campus with hundreds of users hitting the same popular pages, the freed bandwidth is substantial.

---

7. Networking Theory: Why This Works

The Nodal Delay Model

Every packet experiences total nodal delay:

d_nodal = d_proc + d_queue + d_trans + d_prop

Here's how it breaks down for a direct request vs. a proxy cache hit:

Term	Direct (throttled, 15 Mbps link)	Proxy Cache Hit
d_proc	~1 ms	~0.1 ms
d_queue	Grows as load → link capacity	~0 ms (LAN is lightly loaded)
d_trans	L/R = bits ÷ 15 Mbps	L/R on LAN ≈ 0 (100 Mbps)
d_prop	RTT/2 ≈ 25 ms	0 ms (same machine)

The proxy eliminates d_trans and d_prop for cache hits — which are the two biggest contributors when the access link is the bottleneck.

Traffic Intensity

I = La / R

- **L** = object size in bits
- **a** = request arrival rate (req/s)
- **R** = link capacity (15 Mbps)

As **I → 1**, queuing delay → ∞. By serving cache hits locally, EdgeShield reduces the effective **La** hitting the WAN, keeping traffic intensity well below 1. This benefits every user on the LAN simultaneously — not just the ones getting cached responses.

DNS Sinkholing Effect

When doubleclick.net resolves to 0.0.0.0, the client attempts a TCP connection to that IP, which immediately fails. The result: **zero bytes cross the access link** for those domains. This reduces both WAN traffic and competing noise on the shared pipe, which lowers queuing delays for legitimate requests.

---

8. Challenges I Faced

1. Building the DNS Packet Parser from Scratch The hardest part was implementing wire-format DNS parsing using Python's struct module without any DNS library. Getting the TX-ID swap right (so the client sees its own query ID in

the cached response) took multiple debugging sessions. A misaligned byte offset causes the client to silently discard the response — no error, just nothing.

2. Network Port 53 Blocked On my machine, the system `resolved` service held port 53, and my campus network blocked outbound port 53 to external servers. I had to use port 5300 and forward to `127.0.0.53:53` instead of `1.1.1.1:53`. This required setting the `EDGESHIELD_UPSTREAM` environment variable before running the resolver.

3. `Connection: close` was Critical Without `Connection: close` in benchmark requests, the proxy held the connection open waiting for more requests, causing the benchmark to hang indefinitely. This was a subtle bug that cost me a lot of debugging time before I found it.

4. `tc` Throttling Only Affects External Traffic When I first set up link throttling and ran benchmarks, the proxy cache hits were still blazing fast. This is actually correct behaviour — `netem` only affects the external interface, not loopback. Cache hits served over `127.0.0.1` correctly bypass the throttle. The throttle is meant to make WAN requests slower, not to punish cache hits.

5. Figure 3 is a Model, Not a Measurement `fig3_delay_model.png` computes theoretical `d_trans + d_prop` for different object sizes using the given link parameters. I initially tried to validate it against benchmark numbers, but it's explicitly a mathematical illustration of the delay model equations — not drawn from CSVs.

---

9. Limitations & Honest Reflection

It's important to be upfront about what this project doesn't do:

Limitation	Reality
HTTP only	No HTTPS/TLS support. All test URLs must be `http://`. A real proxy would use `CONNECT` tunnelling for HTTPS.
Single client	Only one request at a time — no concurrency. Real caches handle dozens of simultaneous connections.
Loopback "LAN"	Proxy and client are on the same

Limitation	Reality
	machine, so true LAN delay is 0. A real setup would have them on different hosts.
DNS not integrated	The proxy uses the OS resolver, not EdgeShield DNS. A real system would route proxy DNS through its own resolver.
Cache correctness	Conditional GETs depend on the origin sending a Last-Modified header, which not all servers do.
tc emulation	netem adds a fixed delay — it doesn't simulate realistic competing traffic or TCP congestion dynamics.

10. Experiment Task Checklist

Task	What I Did	Outcome
1	Compatibility check + curl test	Got HTTP 200 response through proxy
2	DNS functional test with dig	All 4 resolution paths logged in dns_log.csv
3	Baseline benchmark (run_baseline/)	100 HTTP rows, 40 DNS rows, 0 errors
4	Enabled link throttle with tc	ping example.com showed ~50 ms added; loopback unaffected
5	Throttled benchmark + figures	3 PNG figures generated in run_throttled/
6	Results tables computed from CSVs	Mean/median/p95/min/max per mode calculated
7	Wireshark captures	Captured and annotated proxy vs. direct traffic patterns
8	This report	Written entirely from real

Task	What I Did	Outcome
		CSV data, no placeholders

11. Conclusion

This project took a concept from the textbook — traffic intensity, access link bottlenecks, edge caching — and made it something I could actually measure, observe, and reason about with real numbers.

The **118.80× speedup** on cache hits isn't just a cool statistic. It directly translates to bandwidth freed on the shared access link, which benefits every user on the LAN simultaneously. The **DNS sinkhole** eliminating ad traffic is similarly elegant — by returning `0.0.0.0` in under 0.4 ms, we prevent the client from ever attempting a WAN connection for those domains.

What I really took away from this: **the edge is a leverage point**. Small decisions at the edge — what to cache, what to block, what to resolve locally — have outsized effects on the shared bottleneck. EdgeShield is a tiny implementation of a principle that CDNs and enterprise edge proxies use at massive scale.

If I were to extend this project, I'd add: HTTPS CONNECT tunnelling, concurrent client handling with `asyncio`, actual TTL-based HTTP cache expiry (instead of the 30-second age check), and integration between the DNS resolver and the proxy so they share a unified upstream stack.

References

1. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th Edition — Chapters 2 (Application Layer), 4 (Network Layer)
2. RFC 1035 — Domain Names: Implementation and Specification
3. RFC 7234 — HTTP/1.1 Caching
4. Linux `tc-netem` man page — Network Emulation with Linux Traffic Control
5. Python `struct` module documentation — Binary Data Structures

Report prepared by Siddhant Kumar Singh (Roll No. 2024109296) Computer Networks Case Study — EdgeShield Project